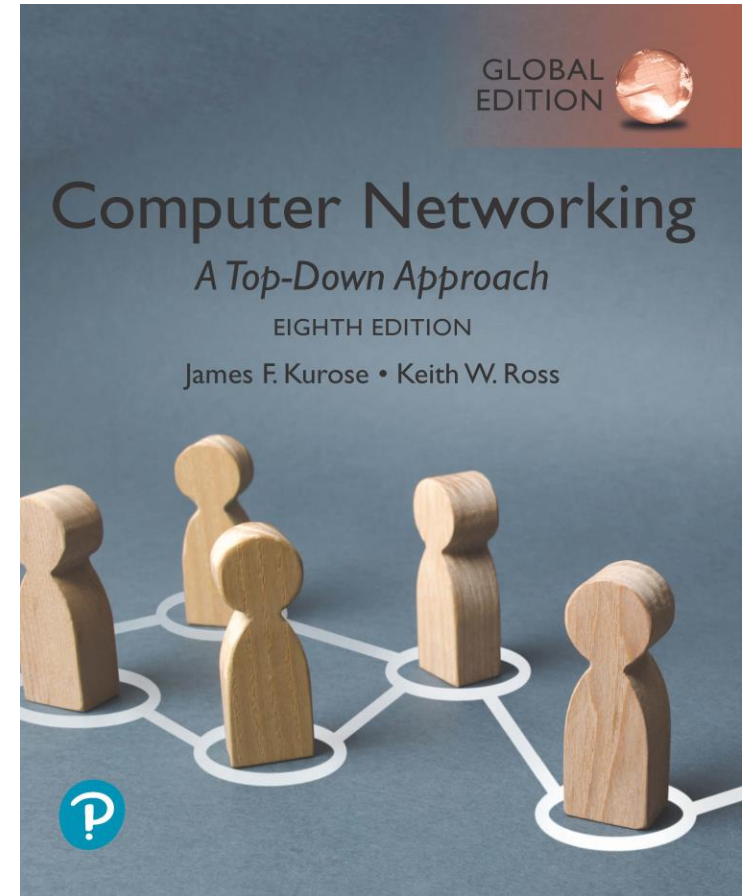


Chapter 8

Security



Computer Networking: A Top-Down Approach

8th edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

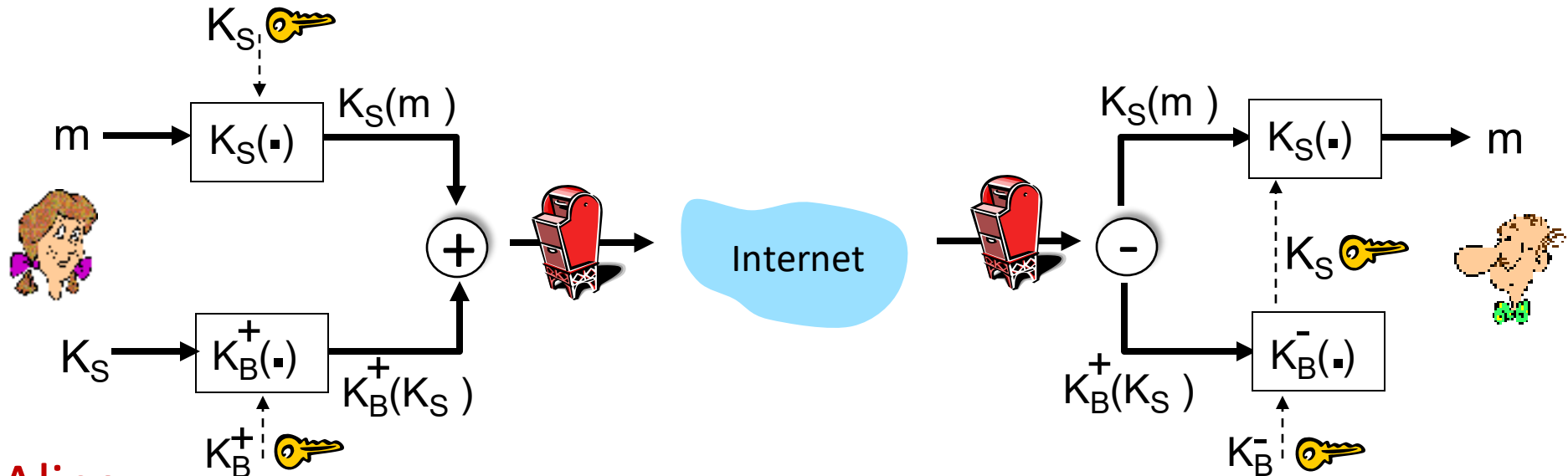
Chapter 8 outline

- What is network security?
- Principles of cryptography
- Authentication, message integrity
- **Securing e-mail**
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Secure e-mail: confidentiality

Alice wants to send *confidential* e-mail, m , to Bob.

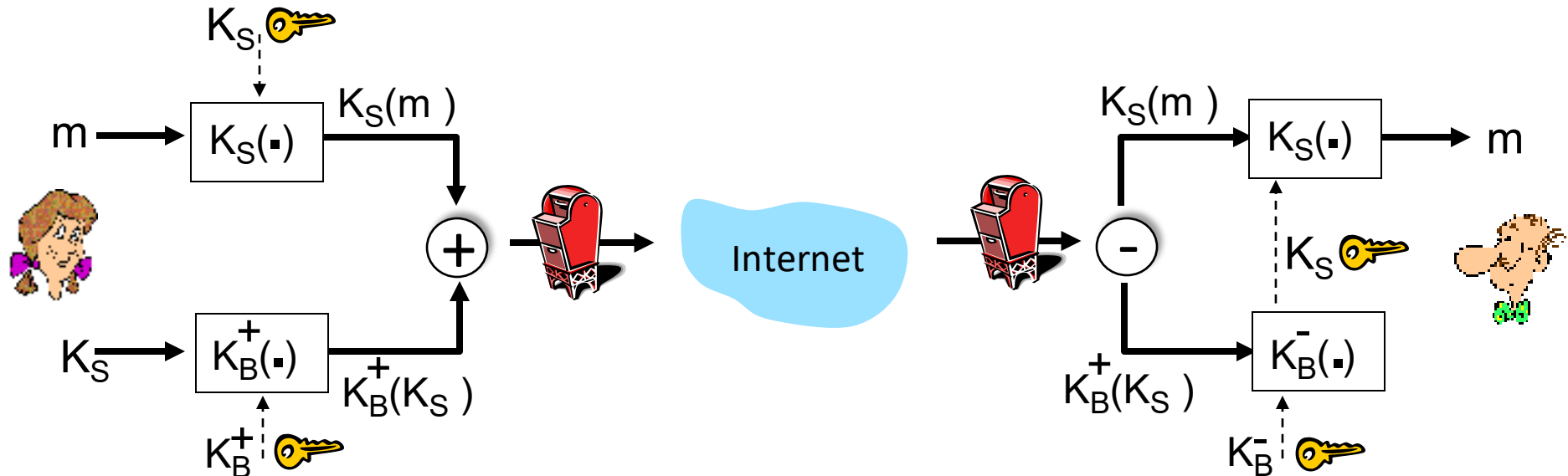


Alice:

- generates random *symmetric* private key, K_S
- encrypts message with K_S (for **efficiency**)
- also encrypts K_S with Bob's public key
- sends both $K_S(m)$ and $K_B^+(K_S)$ to Bob

Secure e-mail: confidentiality (more)

Alice wants to send *confidential* e-mail, m , to Bob.

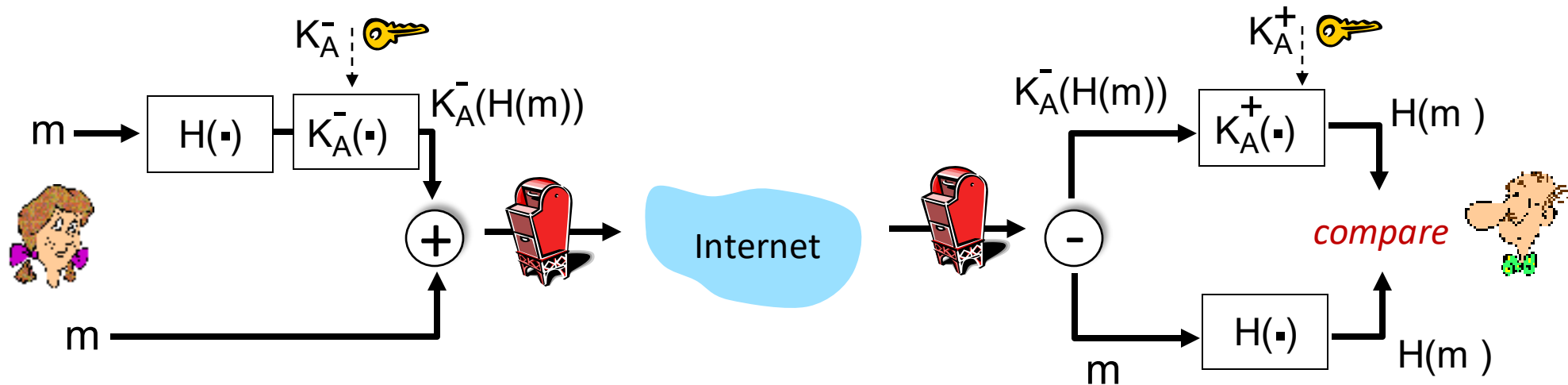


Bob:

- uses his private key to decrypt and recover K_S
- uses K_S to decrypt $K_S(m)$ to recover m

Secure e-mail: integrity, authentication

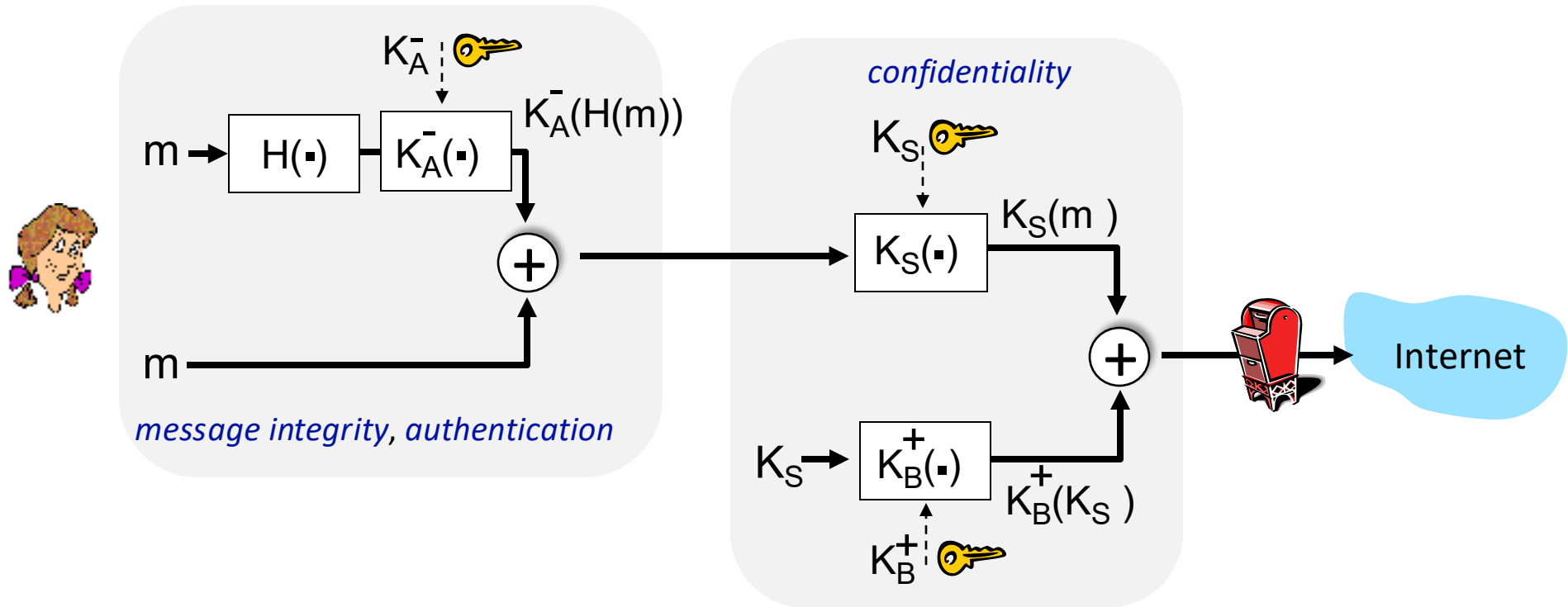
Alice wants to send m to Bob, with *message integrity, authentication*



- Alice digitally signs hash of her message with her private key, providing integrity and authentication
- sends both message (in the clear) and digital signature

Secure e-mail: integrity, authentication, confidentiality

Alice sends m to Bob, with *confidentiality, message integrity, authentication*



Alice uses three keys: her private key, Bob's public key, new symmetric key

What are Bob's complementary actions?

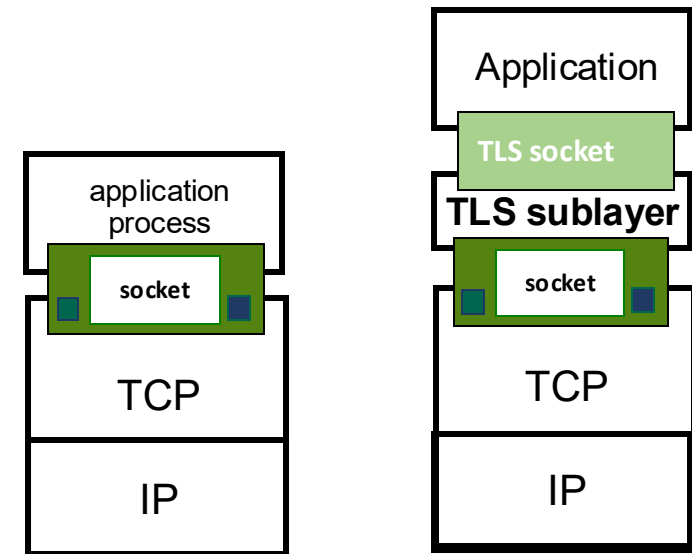
Chapter 8 outline

- What is network security?
- Principles of cryptography
- Authentication, message integrity
- Securing e-mail
- **Securing TCP connections: TLS**
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



TCP and TLS

- Widely deployed security protocol
 - supported by almost all browsers, web servers
 - Adding security to transport layer
- TLS provides an API that *any* application can use
 - Secures a stream of ANY data



Transport-layer security (TLS)

- widely deployed security protocol above the transport layer
 - supported by almost all browsers, web servers: **https** (**port 443**)
- provides:
 - **confidentiality**: via *symmetric encryption*
 - **integrity**: via *cryptographic hashing*
 - **authentication**: via *public key cryptography*

} *all techniques we have studied!*
- history:
 - early research, implementation: secure network programming, secure sockets
 - secure socket layer (SSL) deprecated [2015]
 - TLS 1.0 based on SSL 3.0 (but not quite the same)
 - TLS 1.3: RFC 8846 [2018]

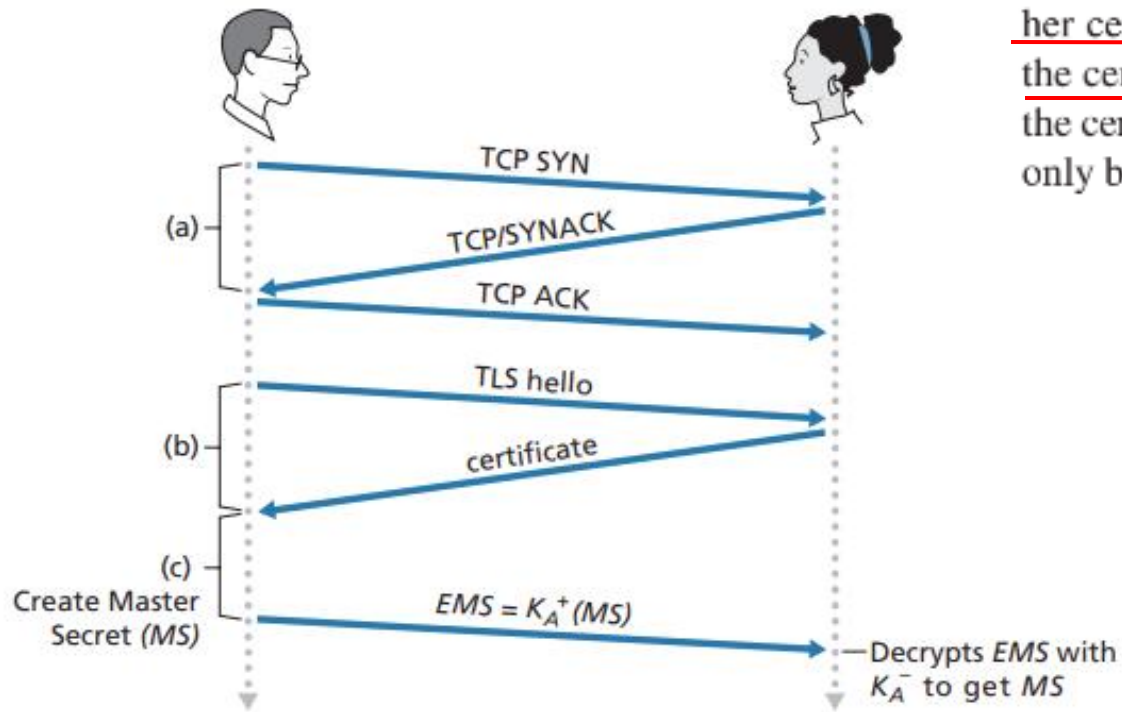
Transport-layer security: what's needed?

- let's *build* a **toy TLS protocol**, *t-tls*, to see what's needed!
- we've seen the “pieces” already:
 - **handshake**: Alice, Bob use their certificates, private keys to authenticate each other, exchange or create shared secret
 - **key derivation**: Alice, Bob use shared secret to derive set of keys
 - **data transfer**: stream data transfer: data as a series of **records**
 - not just one-time transactions
 - **connection closure**: special messages to securely close connection

t-tls: initial handshake (see pg. 676)

Handshake

During the handshake phase, Bob needs to (a) establish a TCP connection with Alice, (b) verify that Alice is *really* Alice, and (c) send Alice a master secret key, which will be used by both Alice and Bob to generate all the symmetric keys they need for the TLS session. These three steps are shown in Figure 8.25. Note that once the TCP connection is established, Bob sends Alice a hello message. Alice then responds with her certificate, which contains her public key. As discussed in Section 8.3, because the certificate has been certified by a CA, Bob knows for sure that the public key in the certificate belongs to Alice. Bob then generates a Master Secret (MS) (which will only be used for this TLS session), encrypts the MS with Alice's public key to create



EMS = Encrypted Master Secret

t-tls: cryptographic keys (fortsatt "toy"-tls)

- considered bad to use same key for more than one cryptographic function
 - different keys for message authentication code (MAC) and encryption
- E_B = session encryption key for data sent from Bob to Alice
- M_B = session HMAC key for data sent from Bob to Alice, where HMAC [RFC 2104] is a standardized hashed message authentication code (MAC) that we encountered in section 8.3.2
- E_A = session encryption key for data sent from Alice to Bob
- M_A = session HMAC key for data sent from Alice to Bob

Alice and Bob each generate the four keys from the MS. This could be done by simply slicing the MS into four keys. (But in *reality* TLS it is a little more complicated, as we'll see.) At the end of the key derivation phase, both Alice and Bob have all four keys. The two encryption keys will be used to encrypt data; the two HMAC keys will be used to verify the integrity of the data.

t-tls: encrypting data

- recall: TCP provides data *byte stream* abstraction
- Q: can we encrypt data in-stream as written into TCP socket?
 - A: where would MAC go? If at end, no message integrity until all data received and connection closed!
 - solution: break stream in series of “records”
 - each client-to-server record carries a MAC
 - receiver can act on each record as it arrives
- t-tls record encrypted using symmetric key, K_c , passed to TCP:



Note K_c is either EB or EA - MAC is either MB or MA

TLS: 1.3 cipher suite (kom i 2018)

- “cipher suite”: algorithms that can be used for key generation, encryption, MAC, digital signature
- TLS: 1.3 (2018): more limited cipher suite choice than TLS 1.2 (2008)
 - only 5 choices, rather than 37 choices
 - *requires* Diffie-Hellman (DH) for key exchange, rather than DH or RSA
 - combined encryption and authentication algorithm (“authenticated encryption”) for data rather than serial encryption, authentication
 - HMAC uses SHA (256 or 284) cryptographic hash function